

OAI 5G NR LEO NTN End-to-End Deployment Guide

Target: transparent LEO satellite, 600 km circular orbit, band n254, 5 MHz, 15 kHz SCS, FDD, rfsimulator.

Stack: Open5GS core, OAI nr-softmodem gNB, OAI nr-uesoftmodem UE, single host.

Source: Duranta OpenAirInterface, `develop` branch.

Part 0. Host preparation

0.1 Operating system

Ubuntu 22.04 LTS or 24.04 LTS. Both are supported by the Duranta fork. 24.04 is the safer choice if you are starting fresh, since the OAI UE application note targets 24.04.3.

0.2 CPU requirement

OAI PHY needs AVX2. Confirm before anything else:

```
lscpu | grep -o 'avx2\|avx512' | sort -u
```

If AVX2 is absent, stop. The build will succeed and the softmodem will fail at runtime in ways that look like PHY bugs.

0.3 Resources

rfsimulator with 25 PRB at 15 kHz SCS is light. Four cores and 8 GB RAM is workable. Eight cores and 16 GB is comfortable and lets you run core, gNB, and UE plus logging without contention.

No SDR hardware is needed. No UHD, no USRP. This is the main reason rfsimulator is the right instrument for this study.

0.4 Performance tuning

Not strictly required for rfsimulator, but timing loops behave more consistently with the governor pinned:

```
sudo apt install -y linux-tools-common linux-tools-$(uname -r) cpufrequtils  
sudo cpupower frequency-set -g performance
```

Disable any aggressive C-state or turbo variability if you see jitter in TA commands later. Do not chase this on day one.

0.5 Base packages

```
sudo apt update
sudo apt install -y git build-essential cmake ninja-build pkg-config \
    libssl-dev libconfig-dev libatlas-base-dev libblas-dev \
    iproute2 iputils-ping iperf3 net-tools tcpdump curl gnupg
```

Part 1. Open5GS core

Open5GS over free5GC for this work. free5GC needs the `gtp5g` out-of-tree kernel module, which breaks on kernel upgrades and adds a failure mode that has nothing to do with NTN. Use free5GC only if you have a specific reason.

1.1 MongoDB

Open5GS stores subscriber data in MongoDB. On 24.04 (noble):

```
curl -fsSL https://www.mongodb.org/static/pgp/server-8.0.asc | \
    sudo gpg -o /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor

echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ]
https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 multiverse" | \
    sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list

sudo apt update
sudo apt install -y mongodb-org
sudo systemctl enable --now mongod
```

On 22.04, replace `noble` with `jammy` in the repo line.

Verify:

```
systemctl status mongod --no-pager
```

If the MongoDB repo step fails for any reason, fall back to the dockerised Open5GS in `ocudu/docker` or the official Open5GS Docker Compose. The rest of this guide is unaffected, only the config file paths change.

1.2 Open5GS packages

```
sudo add-apt-repository -y ppa:open5gs/latest
sudo apt update
sudo apt install -y open5gs
```

This installs and starts all network functions as systemd units: `open5gs-amfd`, `open5gs-smfd`, `open5gs-upfd`, `open5gs-ausfd`, `open5gs-udmd`, `open5gs-udrd`, `open5gs-nrfd`, `open5gs-pcfd`, `open5gs-nssfd`, `open5gs-bsfd`, `open5gs-scpd`.

Check them all:

```
systemctl list-units 'open5gs-*' --no-pager
```

1.3 WebUI for subscriber provisioning

```
sudo apt install -y nodejs npm
curl -fsSL https://open5gs.org/open5gs/assets/webui/install | sudo -E bash -
```

The WebUI listens on `http://localhost:9999`. Default login is `admin` with password `1423`. Change it or bind it to loopback only if the machine is not isolated.

1.4 Determine the PLMN the gNB will use

Do not assume. Read it out of the gNB config you are about to run:

```
cd ~/openairinterface5g # after Part 2 clone
grep -E "mcc|mnc|tracking_area_code|sst|sd" \
  ci-scripts/conf_files/gnb.sa.band254.u0.25prb.rfsim.ntn-leo.conf
```

Almost certainly MCC 001, MNC 01, TAC 1, SST 1. Whatever it prints is authoritative. Everything in the next two steps must match it.

1.5 Configure the AMF

Edit `/etc/open5gs/amf.yaml`. Three blocks matter.

The NGAP bind address, which the gNB will connect to:

```
amf:
  ngap:
    server:
      - address: 127.0.0.5
```

For a single host this is fine. If gNB and core are on separate machines, set this to the core machine's LAN address.

The GUAMI, TAI, and supported PLMN, which must match section 1.4:

```

guami:
  - plmn_id:
      mcc: 001
      mnc: 01
    amf_id:
      region: 2
      set: 1
tai:
  - plmn_id:
      mcc: 001
      mnc: 01
    tac: 1
plmn_support:
  - plmn_id:
      mcc: 001
      mnc: 01
    s_nssai:
      - sst: 1

```

A PLMN or TAC mismatch produces an NG Setup Failure with cause "unknown PLMN", which is a clean and obvious error. A slice mismatch is subtler: registration completes and the PDU session then fails, so the UE reaches RRC connected and never gets an IP.

Restart after editing:

```

sudo systemctl restart open5gs-amfd
sudo journalctl -u open5gs-amfd -f

```

1.6 Provision the subscriber

Read the credentials out of the UE config so you provision exactly what the UE will present:

```

grep -A6 "uicc0" targets/PROJECTS/GENERIC-NR-5GC/CONF/ue.conf

```

Typical OAI defaults:

Field	Value
IMSI	001010000000001
K	fec86ba6eb707ed08905757b1bb44b8f
OPc	C42449363BBAD02B66D16BC975D77CC1
DNN	oai
SST	1

Enter these in the WebUI under Subscriber, Add. Set the DNN and SST to match the UE config exactly.

The DNN is a frequent trap. If the UE requests `oai` and the subscriber is provisioned for `internet`, registration succeeds and the PDU session is rejected. Either change the subscriber's APN or change

`dnn` in the UE's `pdu_sessions` block. They must agree.

1.7 Verify the data plane interface

```
ip addr show ogstun
```

Expect `10.45.0.1/16`. If the interface is missing, the UPF did not start:

```
sudo journalctl -u open5gs-upfd -n 50 --no-pager
```

1.8 Enable forwarding and NAT

Needed only if you want the UE to reach the internet. Ping and iperf against the core work without it.

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -s 10.45.0.0/16 ! -o ogstun -j MASQUERADE
```

Make permanent via `/etc/sysctl.conf` and `iptables-persistent` if you want it to survive reboot.

1.9 Core checkpoint

Before moving on, all Open5GS units are active, `ogstun` exists with `10.45.0.1/16`, and the subscriber appears in the WebUI. The core is now finished. Nothing in the NTN path touches it again.

Part 2. Build OAI

2.1 Clone

```
cd ~
git clone https://github.com/duranta-project/openairinterface5g.git
cd openairinterface5g
git checkout develop
git log -1 --format="%H %ad %S"
```

Record that commit hash. You will need it for reproducibility in any write-up, and NTN code in this area moves.

2.2 Read the build doc first

```
less doc/BUILD.md
```

The flags below are the standard OAI build interface and are stable across forks, but the Duranta fork may have added or renamed targets. If `BUILD.md` contradicts anything here, follow `BUILD.md`.

2.3 Install dependencies

```
source oaienv
cd cmake_targets
./build_oai -I
```

This pulls the full dependency set including UHD. It takes 10 to 30 minutes and needs sudo. Run it once per machine.

You do not need UHD for rfsimulator, but `-I` installs it anyway and it is harmless.

2.4 Build gNB, UE, and rfsimulator

```
./build_oai -w SIMU --gNB --nrUE --ninja
```

`-w SIMU` is what builds the rfsimulator device library. Without it you get no `--rfsim` support and no channel models, which is the entire mechanism this deployment depends on.

Expect 15 to 40 minutes depending on cores.

2.5 Verify the build

```
ls -la ran_build/build/nr-softmodem ran_build/build/nr-uesoftmodem
ls -la ran_build/build/librfsimulator.so
```

All three must exist. If `librfsimulator.so` is missing, the `-w SIMU` flag did not take effect. Rebuild rather than working around it.

Confirm the channel models are compiled in:

```
strings ran_build/build/librfsimulator.so | grep -i "SAT_LEO"
```

You should see `SAT_LEO_TRANS` and `SAT_LEO_REGEN`. If not, your checkout predates the LEO channel models and you need a newer commit.

Part 3. Understand the radio configuration

Worth five minutes before editing anything, because the numbers look arbitrary and are not.

3.1 Band n254

n254 is an S-band NTN band, FDD. Downlink 2483.5 to 2500 MHz, uplink 1610 to 1626.5 MHz.

The UE command line carries `-C 2488400000` and `--CO -873500000`. The first is the DL centre frequency, 2488.4 MHz. The second is the UL offset. 2488.4 minus 873.5 gives 1614.9 MHz UL, which sits inside the n254 uplink range. The offset is negative because in this band the uplink is below the downlink, which is unusual and is why the parameter has to be given explicitly.

`-r 25` is 25 PRB, which at 15 kHz SCS is 5 MHz. `--numerology 0` is 15 kHz SCS. `--ssb 60` places the SSB starting at subcarrier 60 within the 300 subcarrier carrier, which centres the 240 subcarrier SSB

block.

3.2 The two channel models

`SAT_LEO_TRANS` is a transparent satellite with the gNB on the ground. The feeder link is in the delay budget.

`SAT_LEO_REGEN` is a regenerative satellite with the gNB on board. No feeder link.

Both simulate delay and Doppler for a 600 km circular orbit, following the MATLAB `dopplerShiftCircularOrbit` model. This guide uses `SAT_LEO_TRANS`.

3.3 Where the LEO numbers come from

The gNB config carries these LEO values:

Parameter	Config value	Physical value
<code>cellSpecificOffset-r17</code>	40	40 slots at 15 kHz
<code>ta-Common-r17</code>	4634000	18.87 ms
<code>ta-CommonDrift-r17</code>	-230000	-46 µs/s
<code>positionY-r17</code>	-2166908	-2816980.4 m
<code>positionZ-r17</code>	4910784	6384019.2 m
<code>velocityVY-r17</code>	115246	6914.76 m/s
<code>velocityVZ-r17</code>	50853	3051.18 m/s

The granularities are $4.072\text{e-}3\text{ }\mu\text{s}$ for `ta-Common`, $0.2\text{e-}3\text{ }\mu\text{s/s}$ for `ta-CommonDrift`, 1.3 m for position, and 0.06 m/s for velocity.

The UE flags derive from the same geometry. Frequency offset is $f_c * v_{\text{radial}} / c$. At 2488.4 MHz with 6914.76 m/s that gives roughly 57.3 kHz, which is the `--initial-fo 57340` value.

Time drift is range rate divided by c . The -46 µs/s figure corresponds to about 13.8 km/s of range rate, roughly twice the service link velocity, because the transparent architecture puts the feeder link into the same delay budget. Verify this factor against your own logs before relying on it. It is a config convention in this example, not a derived constant.

The key point for your experiments: the gNB `ta-CommonDrift-r17` of -46 µs/s and the UE `--ntn-initial-time-drift -46` agreeing is an assumption baked into the shipped example. Breaking that agreement deliberately is the most productive sweep available to you.

Part 4. Configure the gNB

4.1 Locate the files

```
cd ~/openairinterface5g
ls -la ci-scripts/conf_files/gnb.sa.band254.u0.25prb.rfsim.ntn-leo.conf
ls -la targets/PROJECTS/GENERIC-NR-5GC/CONF/channelmod_rfsimu_LEO_satellite.conf
```

Both should exist in a current checkout.

4.2 Verify the LEO ephemeris is active

The GEO config file has the LEO block commented out. The LEO config file should have it live. Confirm rather than assume:

```
grep -E "Koffset_r17|ta-Common-r17|ta-CommonDrift|position[XYZ]-r17|velocityV[XYZ]-r17" \
  ci-scripts/conf_files/gnb.sa.band254.u0.25prb.rfsim.ntn-leo.conf
```

Every one of those lines must be uncommented, and the values must match the LEO column in section 3.3. If you see `ta-Common-r17 = 58629666`, you are looking at GEO values and you have the wrong file or the wrong block enabled.

4.3 Verify the channel model reaches the gNB

The gNB applies the channel model to the uplink. The UE applies it to the downlink. Both sides need it. An asymmetric setup fails in ways that look like a PHY bug.

```
grep -E "@include|channelmod|modellist|SAT_LEO|options" \
  ci-scripts/conf_files/gnb.sa.band254.u0.25prb.rfsim.ntn-leo.conf
```

You need either an inline `channelmod` section naming `SAT_LEO_TRANS`, or an `@include` of the channelmod file.

Include path resolution is a real trap. The gNB config lives in `ci-scripts/conf_files/` and the channelmod file lives in `targets/PROJECTS/GENERIC-NR-5GC/CONF/`. If the include is a bare filename, it will not resolve across those directories. The robust fix is to copy the channelmod file next to each config that includes it:

```
cp targets/PROJECTS/GENERIC-NR-5GC/CONF/channelmod_rfsimu_LEO_satellite.conf \
  ci-scripts/conf_files/
```

Then the bare `@include "channelmod_rfsimu_LEO_satellite.conf"` resolves regardless of how the parser handles relative paths.

Also confirm `chanmod` is in the rfsimulator options, either as `options = ("chanmod");` in the conf `rfsimulator` section or via the command line flag. The UE command below passes it explicitly, so do the same on the gNB side if the conf does not already have it.

4.4 SIB19 scheduling

This is LEO-specific and did not matter in your GEO run.

`ntn-ULSyncValidityDuration-r17` is the maximum time the UE may keep using assistance information without refreshing it. For LEO the reference value is about 5 s, against 240 s for GEO. Since OAI has not implemented epoch time, the UE starts this timer from the TTI where SIB19 is received.

So SIB19 must be broadcast well inside 5 s or the UE loses sync a few seconds after acquiring it. Check the SI scheduling:


```
grep -A20 "si_SchedulingInfo\|sib19\|si_Periodicity\|si_WindowLength" \
  ci-scripts/conf_files/gnb.sa.band254.u0.25prb.rfsim.ntn-leo.conf
```

If the SIB19 period is longer than about 2 s, shorten it. A period well under the validity duration gives the UE margin.

4.5 Timers

The shipped LEO config should already have extended timers. Confirm:

```
grep -A10 "TIMERS" ci-scripts/conf_files/gnb.sa.band254.u0.25prb.rfsim.ntn-leo.conf
```

Expect `sr_ProhibitTimer_v1700`, `t300`, `t301`, and `t319` extended. LEO needs less extension than GEO, since RTT is roughly 40 ms rather than 480 ms, but the defaults are sized for terrestrial and are too tight either way.

4.6 HARQ

For LEO, leave HARQ enabled. `disable_harq = 1` is a GEO measure. At 40 ms RTT, HARQ feedback still arrives inside a usable window, and disabling it breaks compatibility with UEs that do not support the Release 17 feature.

If you want more HARQ processes:

```
num_dlharq = 32;
num_ulharq = 32;
```

This is one of your sweep axes later. Leave it at defaults for first attach.

4.7 Point the gNB at the AMF

```
grep -E "amf_ip_address|GNB_INTERFACE_NAME_FOR_NG|GNB_IPV4_ADDRESS_FOR_NG" \
  ci-scripts/conf_files/gnb.sa.band254.u0.25prb.rfsim.ntn-leo.conf
```

`amf_ip_address.ipv4` must be the AMF NGAP address from section 1.5, typically `127.0.0.5`. The NG-C and NG-U interface names should be `lo` for a single host, or your LAN interface for a split setup.

Part 5. Configure the UE

Edit `targets/PROJECTS/GENERIC-NR-5GC/CONF/ue.conf`.

5.1 UICC

```
uicc0 = {
    imsi = "001010000000001";
    key = "fec86ba6eb707ed08905757b1bb44b8f";
    opc = "C42449363BBAD02B66D16BC975D77CC1";
    dnn = "oai";
    nssai_sst = 1;
}
```

These must be byte-identical to what you provisioned in section 1.6.

5.2 Ground position

This is what the UE uses to compute its UE-specific TA. Without it, the UE has no way to separate common TA from its own contribution.

```
position0 = {
    x = 0.0;
    y = 0.0;
    z = 6377900.0;
}
```

This sits on the polar axis, about 21 km below mean earth radius, positioned under the configured orbit. If you move it, the delay and Doppler arithmetic in section 3.3 moves with it and the `--initial-fo` and `--ntn-initial-time-drift` values must be recomputed.

5.3 Channel model

```
@include "channelmod_rfsimu_LEO_satellite.conf"
```

Same path caveat as section 4.3. Confirm the channelmod file is resolvable from the UE config's directory. It should already be at `targets/PROJECTS/GENERIC-NR-5GC/CONF/channelmod_rfsimu_LEO_satellite.conf`, which is the same directory as `ue.conf`, so a bare include works here.

Part 6. Run

Order matters. Core, then gNB, then UE. The gNB is the rfsimulator server and must be listening before the UE starts.

6.1 Core

```
sudo systemctl restart open5gs-amfd open5gs-smfd open5gs-upfd
sudo journalctl -u open5gs-amfd -f
```

Leave that journal open in its own terminal.

6.2 gNB

```
cd ~/openairinterface5g/cmake_targets
sudo ./ran_build/build/nr-softmodem \
  -O ../ci-scripts/conf_files/gnb.sa.band254.u0.25prb.rfsim.ntn-leo.conf \
  --rfsim 2>&1 | tee ~/leo_gnb_$(date +%Y%m%d_%H%M%S).log
```

Wait for NGAP setup before starting the UE.

6.3 UE

```
cd ~/openairinterface5g/cmake_targets
sudo ./ran_build/build/nr-uesoftmodem \
  -O ../targets/PROJECTS/GENERIC-NR-5GC/CONF/ue.conf \
  --band 254 -C 2488400000 --CO -873500000 -r 25 --numerology 0 --ssb 60 \
  --rfsim \
  --rfsimulator.[0].prop_delay 20 \
  --rfsimulator.[0].options chanmod \
  --time-sync-I 0.1 \
  --ntn-initial-time-drift -46 \
  --initial-fo 57340 \
  --cont-fo-comp 2 \
  2>&1 | tee ~/leo_ue_$(date +%Y%m%d_%H%M%S).log
```

Split across machines, add `--rfsimulator.serveraddr <gnb_ip>`.

6.4 The four LEO-only flags

`--ntn-initial-time-drift -46` covers the window before SIB19 arrives, when the UE has no ephemeris but the delay is already moving. GEO never needed this because the delay is static.

`--initial-fo 57340` is the starting Doppler in Hz. Without it the UE searches a frequency range that does not contain the signal.

`--cont-fo-comp 2` continuously compensates DL Doppler and pre-compensates UL Doppler. The assumption is that the local oscillator is accurate and Doppler dominates the frequency error, which holds for this simulation.

`--time-sync-I 0.1` is the integrator gain on the timing loop. First thing to reduce if the TA sequence oscillates.

Optional: `--autonomous-ta` switches TA updates to be derived from DL drift instead of recomputed continuously from SIB19. Comparing the two is one of your best experiments. Do not enable it on the first attach.

Part 7. Verification checkpoints

Work through these in order. Each isolates a layer. Skipping ahead is how you debug the wrong thing.

7.1 gNB

NGAP setup success against the AMF. The Open5GS journal should show the gNB accepted and the count of connected gNBs increment.

Then the cell parameters line, confirming band 254 and 25 PRB.

7.2 UE, in strict order

1. **SSB detection**, PSS and SSS sync. If this never happens, the problem is Doppler search range, not NTN procedures.
2. **MIB decode**.
3. **SIB1 decode**.
4. **SIB19 decode**. This is the gate. If SSB and SIB1 land but SIB19 never appears, the problem is SI scheduling, not NTN sync.
5. **Ephemeris applied**, UE switches from the command-line drift estimate to SIB-derived values.
6. **PRACH** and RA success.
7. **RRCSetup**, RRC connected.
8. **Registration Accept**.
9. **PDU Session Establishment Accept**, with a UE IPv4 in 10.45.x.x.
10. **TUN interface up**, `oaitun_ue1`.

7.3 Data plane

```
ip addr show oaitun_ue1
ping -I oaitun_ue1 10.45.0.1 -c 10
```

Expect roughly 40 ms RTT. If you see 480 ms, the LEO ephemeris block is not active and you are running GEO parameters. Go back to section 4.2.

Throughput:

```
# server, on the host
iperf3 -s -B 10.45.0.1

# client, bound to the UE interface
iperf3 -c 10.45.0.1 -B <UE_IP> -t 30 -i 1
iperf3 -c 10.45.0.1 -B <UE_IP> -t 30 -i 1 -R
```

Part 8. LEO-specific failure modes

These differ from GEO. Pattern-matching from your GEO run will mislead you.

Never acquires, no SSB detection. Doppler outside the UE's search range. Check the sign and magnitude of `--initial-fo`. A sign error presents as total silence, not as degraded performance, which is why it is the most commonly missed mistake here.

SSB and SIB1 decode, SIB19 never arrives. SI scheduling. Section 4.4. Not an NTN sync problem.

Sync acquired, then lost after a few seconds. `ntn-UlSyncValidityDuration-r17` expiring without a SIB19 refresh. Shorten the SIB19 period. This never happened in GEO because the validity duration there is 240 s against about 5 s for LEO.

RACH failures despite correct SSB detection. TA drift during the preamble. Try `--autonomous-ta`. If it helps, the SIB19-derived TA path is not keeping up with the drift.

Timing oscillation in the TA command sequence. Reduce `--time-sync-I` below 0.1.

Attach succeeds then degrades over 30 to 60 s. Expected, not a bug. The ephemeris is static and epoch time is not implemented in OAI, so you are running a linearised snapshot of one point in a pass rather than a real rise-to-set arc. Keep runs short and state this window explicitly in anything you publish.

gNB starts, UE cannot connect to rfsimulator. The gNB must be running first. Check the rfsimulator server port is listening and no stale process holds it.

Registration rejected, unknown IMSI in core log. Subscriber not provisioned, or IMSI mismatch. Section 1.6.

Registration completes, no IP assigned. DNN or slice mismatch. The core log will show a "DNN not supported" or slice warning. Section 1.6.

Part 9. What to instrument from the first successful run

Capture these from run one. Rerunning sweeps because a metric was missing is the standard way these studies lose two weeks.

Per trial: time from UE start to SSB detection, to SIB19 decode, to RRC connected, to PDU session accept. The full TA command sequence from the gNB. Residual frequency offset reported by the UE. MCS and BLER both directions. Ping RTT distribution. iperf throughput both directions.

Attach success is binary per trial, so repetition is mandatory. Twenty to fifty trials per parameter point, scripted. Fewer than that gives you anecdotes.

Part 10. Experiment axes

Ordered by value relative to effort.

Initial access robustness under parameter mismatch. Sweep `--ntn-initial-time-drift` against the true channel drift, and `--initial-fo` against the true Doppler. Map the two-dimensional region where attach succeeds. This produces an acquisition success contour, which is directly useful to system designers and appears to be thin in the published literature for open-source Release 17 NTN implementations. Verify prior art before claiming novelty.

TA strategy comparison. `--autonomous-ta` against SIB19-derived TA. Measure TA error accumulation over time and compare against the roughly 5 s validity window. Self-contained and architecturally relevant.

Transparent versus regenerative. `SAT_LEO_TRANS` against `SAT_LEO_REGEN`. Compare sync behaviour and effective RTT. Maps onto a live 3GPP architecture debate and costs almost nothing to add as a

second axis.

Scheduling and HARQ dimensioning. Sweep `cellSpecificOffset_r17`, `num_dlharq` and `num_ulharq` between 16 and 32, and `disable_harq`. Measure throughput and latency.

Doppler pre-compensation efficacy. Compare `--cont-fo-comp` modes, measure residual UL frequency error at the gNB.

Implementation gap analysis. The unimplemented epoch time is a concrete gap with a measurable consequence. Cataloguing which Release 17 NTN features are implemented, stubbed, or absent is itself a contribution and is what reviewers of a tools paper want to see.

Part 11. Claim boundaries

Be explicit about the instrument in anything you write.

rfsimulator applies delay and Doppler to the sample stream with configurable noise power. There is no 38.811 fading, no scintillation, no realistic path loss. The noise floor is set by `noise_power_dB = -100` in the channelmod config.

Not defensible: BLER or throughput versus SNR presented as representative of a real LEO link. Absolute latency or timing accuracy figures. Anything about a full rise-to-set pass.

Defensible: everything in Part 10. Protocol behaviour, acquisition regions, parameter sensitivity, implementation conformance, and architectural comparison. These are properties of the implementation and the standard, not of the propagation model, which is why the simplified channel does not undermine them.

Reproducibility checklist

Record for any write-up: Duranta commit hash, Ubuntu version, kernel version, Open5GS version, exact config files used, and the full command lines. Publish the configs and sweep scripts alongside the paper. A one-command reproducible LEO testbed is a meaningful part of the contribution.